CHƯƠNG I. KHÁI NIỆM CƠ BẢN

Truy xuất thông tin

ThS. Phạm Đức Cường
cuongpd@ptit.edu.vn

Posts and Telecommunications
Institute of Technology

Học viện Công nghệ Bưu chính Viễn thông

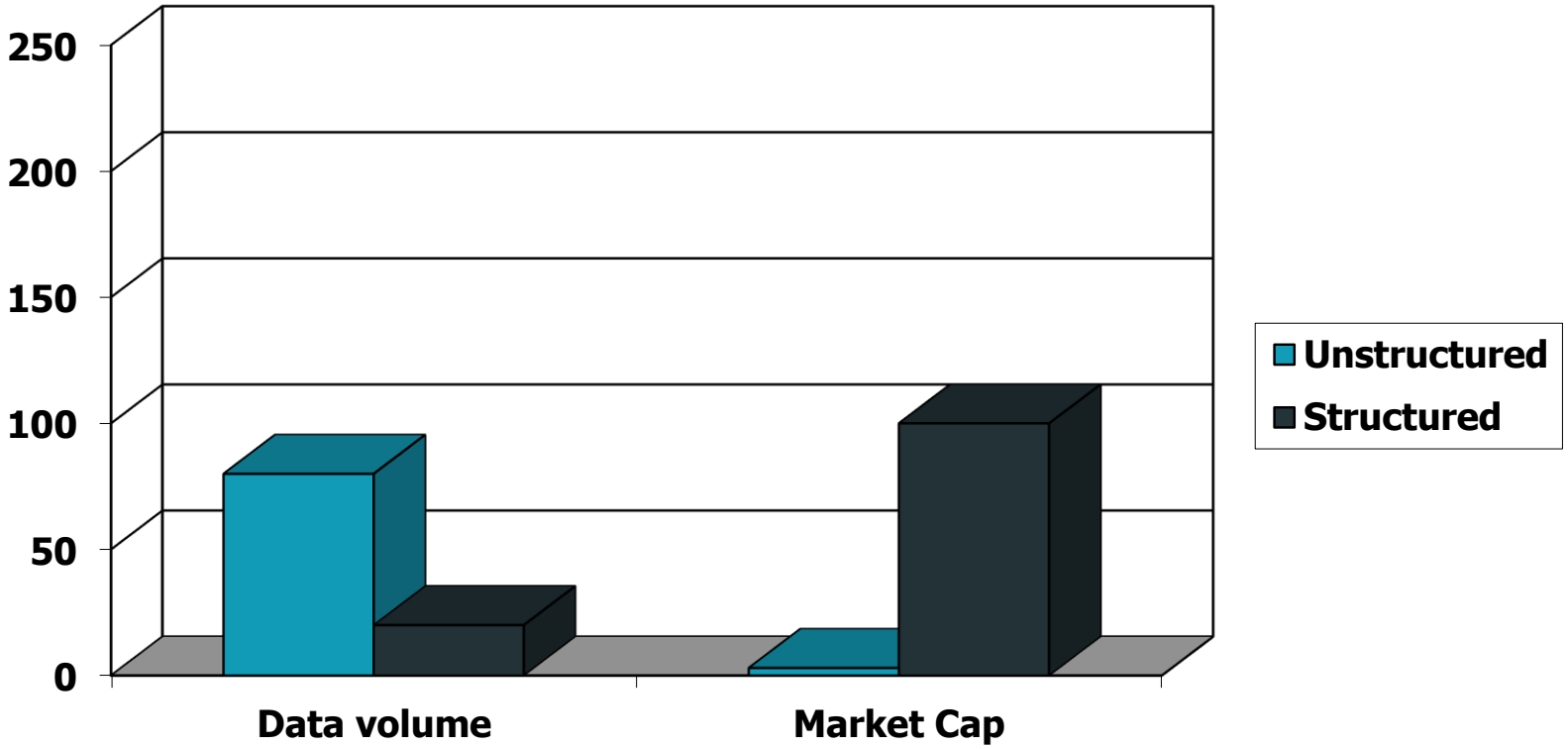
Nội dung

1. Tổng quan về Truy xuất thông tin và Tìm kiếm web
2. Mô hình tìm kiếm và nhu cầu thông tin của người dùng
3. Mô hình truy xuất Boolean và cách xử lý truy vấn logic
4. Chỉ mục ngược và quy trình xây dựng chỉ mục
5. Kỹ thuật xử lý và tối ưu truy vấn
6. Truy vấn cụm từ và chỉ mục vị trí
7. Đánh giá hệ thống tìm kiếm:
   - Độ chính xác (Precision)
   - Độ bao phủ (Recall)
8. Phân biệt dữ liệu có cấu trúc, không cấu trúc và bán cấu trúc
9. Giới thiệu phương pháp xếp hạng kết quả tìm kiếm

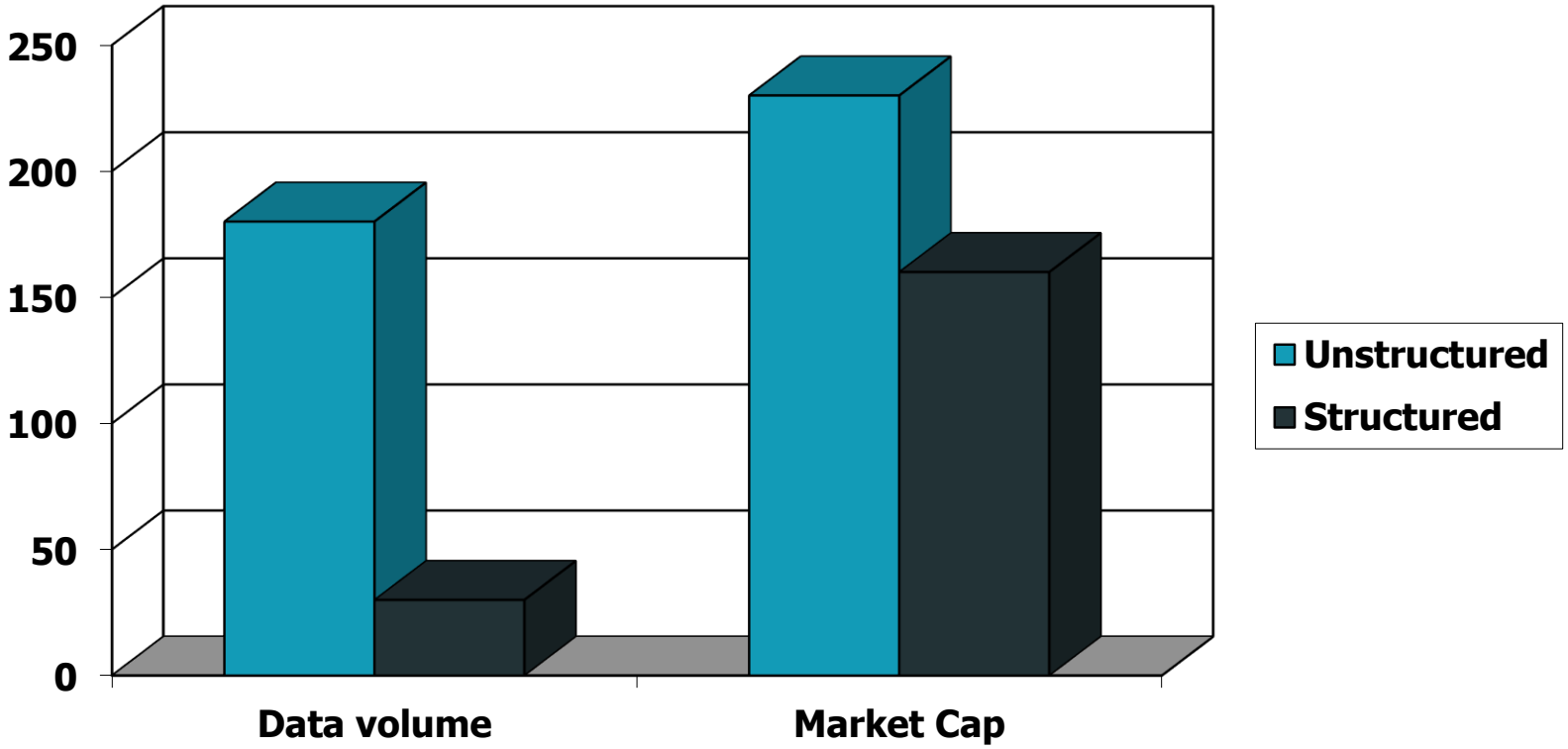
Truy xuất thông tin là gì?

- Truy xuất thông tin - Information Retrieval – IR, là việc tìm kiếm các tài liệu phi cấu trúc (thường là văn bản) thỏa mãn một nhu cầu thông tin từ các bộ sưu tập lớn (thường được lưu trữ trên máy tính)
- Ví dụ:
  - Tìm kiếm trên web (phổ biến nhất)
  - Tìm kiếm E-mail
  - Tìm kiếm trên máy tính xách tay
  - Cơ sở tri thức doanh nghiệp
  - Truy xuất thông tin pháp lý

Dữ liệu phi cấu trúc và có cấu trúc (Giữa thập niên 90)



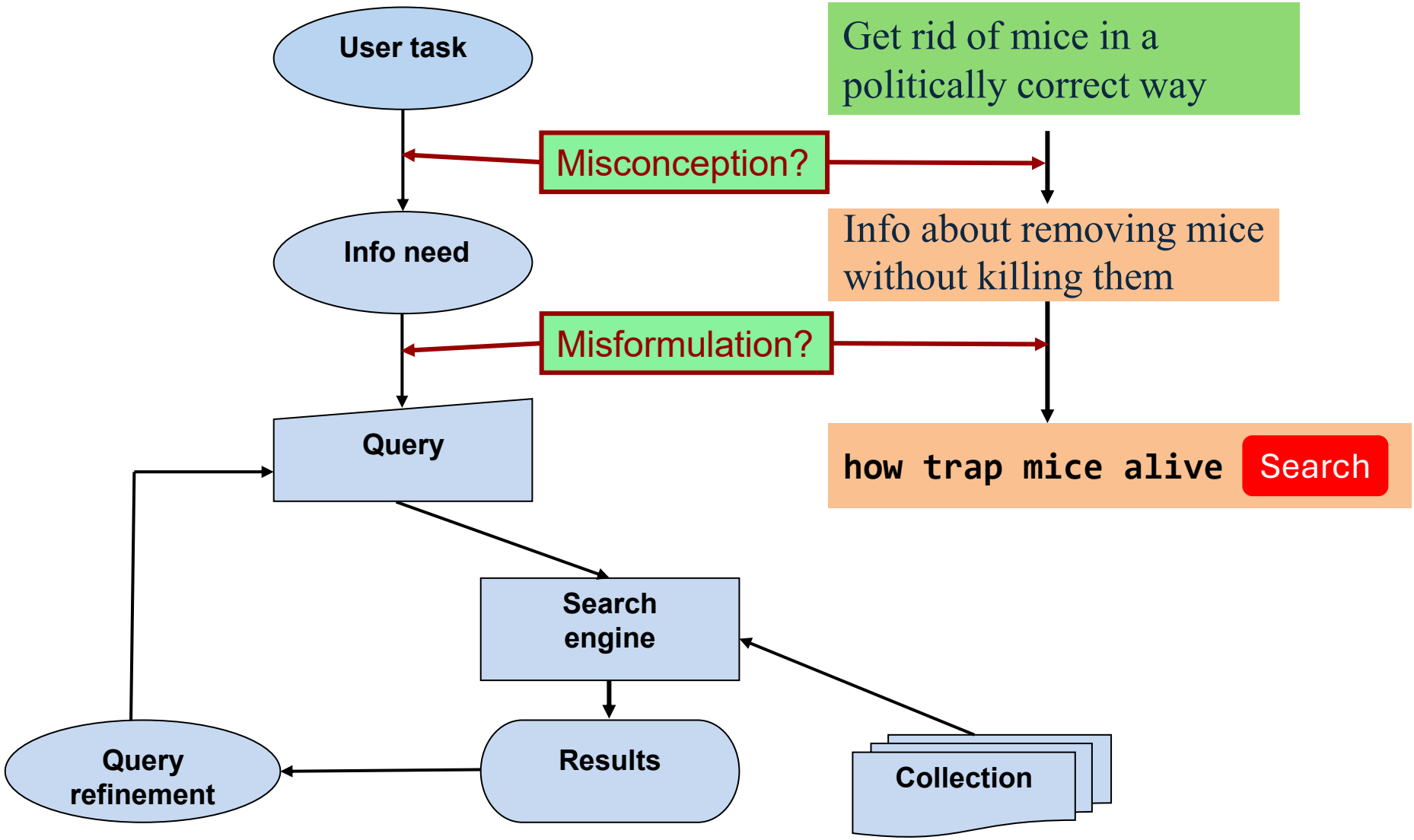
Dữ liệu phi cấu trúc và có cấu trúc (Ngày nay)



Các giả định cơ bản của IR

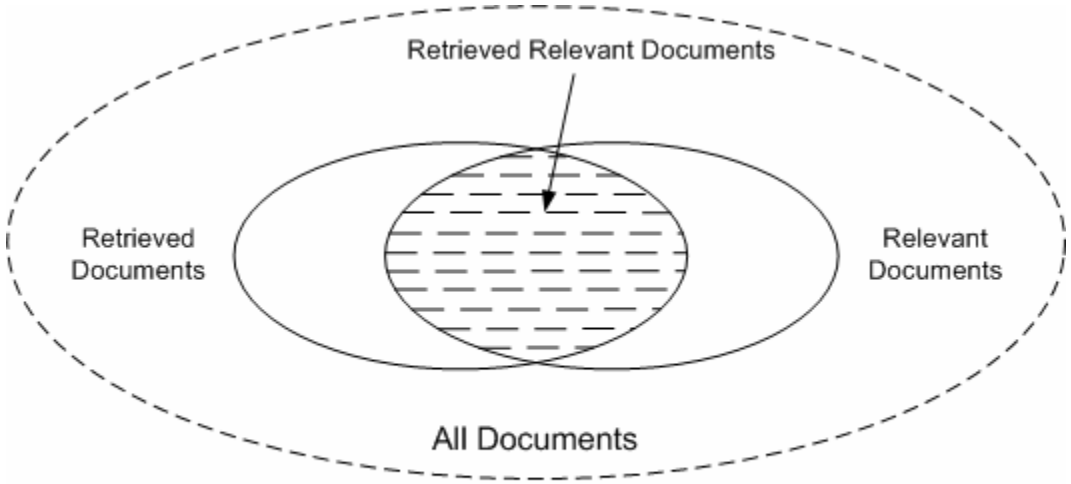
- **Tập tài liệu (Collection):** một tập hợp các tài liệu
  - Giả định: tạm thời coi đây là một bộ sưu tập tĩnh, không thay đổi theo thời gian
- **Mục tiêu:** Truy xuất các tài liệu chứa thông tin liên quan đến nhu cầu của người dùng và giúp người dùng hoàn thành một công việc nào đó

Mô hình tìm kiếm cổ điển



Đánh giá tài liệu được truy xuất

- Độ chính xác (Precision): Tỷ lệ các tài liệu được truy xuất thực sự liên quan đến nhu cầu thông tin của người dùng
- Độ phủ (Recall): Tỷ lệ các tài liệu liên quan có trong bộ sưu tập đã được truy xuất



Recall = # of Retrieved Relevant Documents/# of Relevant Documents

Precision = # of Retrieved Relevant Documents/# of Retrieved Documents

Dữ liệu phi cấu trúc năm 1620

- Những vở kịch nào của Shakespeare chứa các từ **Brutus** VÀ **Caesar** nhưng KHÔNG chứa **Calpurnia**?
- Ta có thể:
  - Dùng lệnh **grep** để tìm trong tất cả các vở kịch của Shakespeare có chứa **Brutus** và **Caesar**
  - Sau đó loại bỏ những vở kịch có chứa **Calpurnia**
- Vì sao cách này không phải là lời giải đúng?
  - Chậm (đối với tập tài liệu lớn)
  - Điều kiện KHÔNG chứa **Calpurnia** khó xử lý
  - Các thao tác khác (ví dụ: tìm từ **Romans** xuất hiện gần **countrymen**) là không khả thi
  - Không hỗ trợ truy xuất có xếp hạng (chọn ra những tài liệu tốt nhất để trả về)

➞ **Các nội dung này sẽ được trình bày trong các bài giảng sau**

Ma trận hiện diện thuật ngữ–tài liệu

- Term-document incidence matrices

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

1 if play contains word, 0 otherwise

Brutus** AND **Caesar** BUT NOT **Calpurnia

Vectơ hiện diện

- Incidence vectors
- Mỗi thuật ngữ có một vectơ 0/1
- Để trả lời truy vấn "**Brutus** AND **Caesar** AND NOT **Calpurnia**": Lấy vectơ của **Brutus**, **Caesar** và vectơ bù của **Calpurnia**, sau đó thực hiện phép toán AND theo bit (bitwise AND)
  - 110100 AND
  - 110111 AND
  - 101111
  - = 100100

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

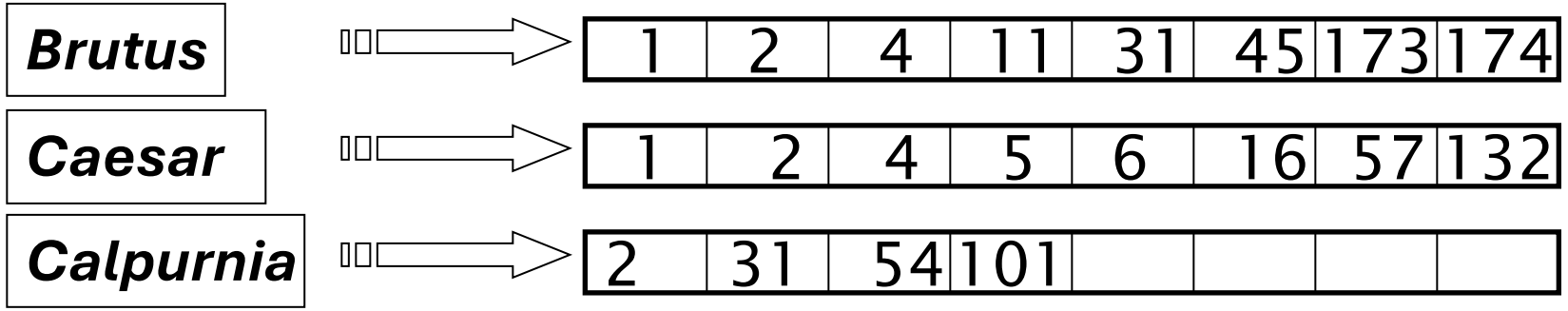
Tập tài liệu lớn hơn

- Giả sử có N = 1 triệu tài liệu, mỗi tài liệu 1000 từ
- Trung bình 6 bytes/từ bao gồm khoảng trắng/dấu câu
  - ➞ 6GB dữ liệu
- Giả sử có M = 500K thuật ngữ khác nhau trong số này
- Ma trận 500K x 1M sẽ có nửa nghìn tỷ ô chứa 0 và 1, nhưng thực tế rất thưa thớt (sparse), chỉ có tối đa một tỉ số 1, chủ yếu là số 0
- Giải pháp tốt hơn là gì?
  - Chỉ ghi lại các vị trí có số 1

Tại sao?

Chỉ mục ngược

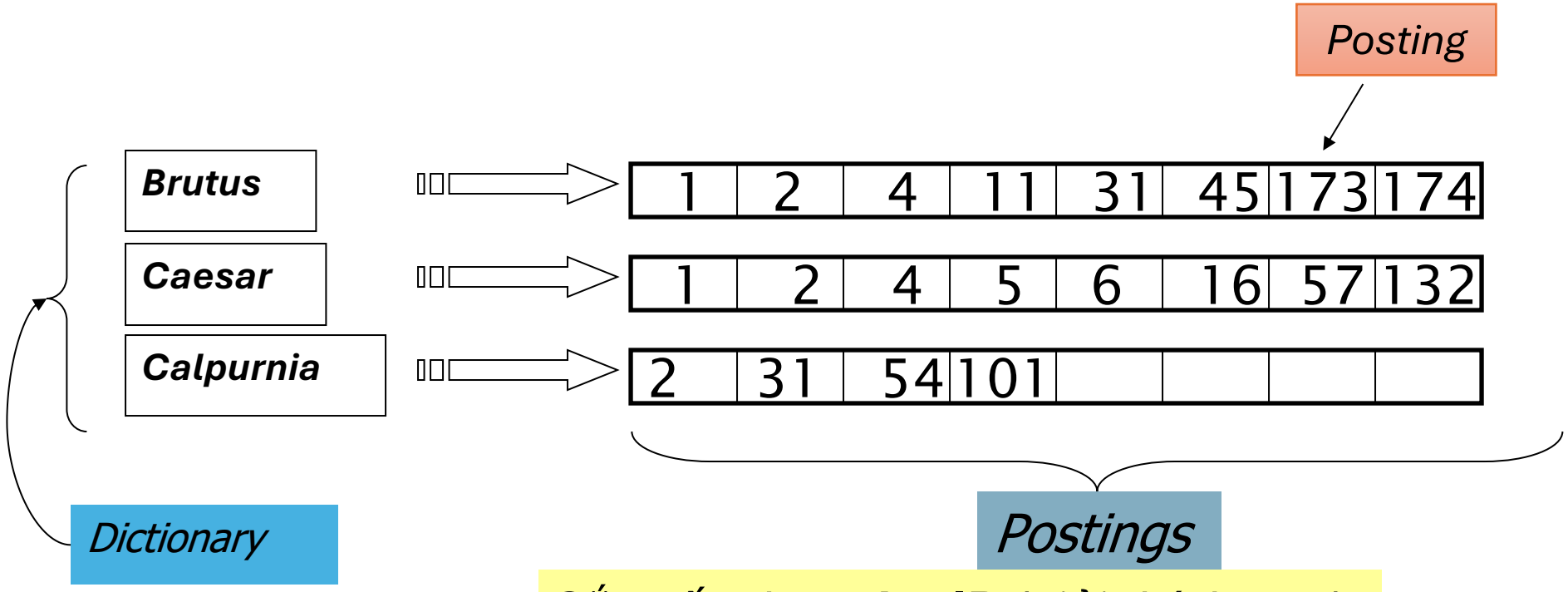
- Inverted index
- Với mỗi thuật ngữ **t**, lưu trữ một danh sách tất cả các tài liệu chứa **t**
  - Định danh mỗi tài liệu bởi một **docID** – một số sêri tài liệu
- Có thể sử dụng mảng kích thước cố định?



Điều gì sẽ xảy ra nếu thêm từ **Caesar** vào tài liệu 14?

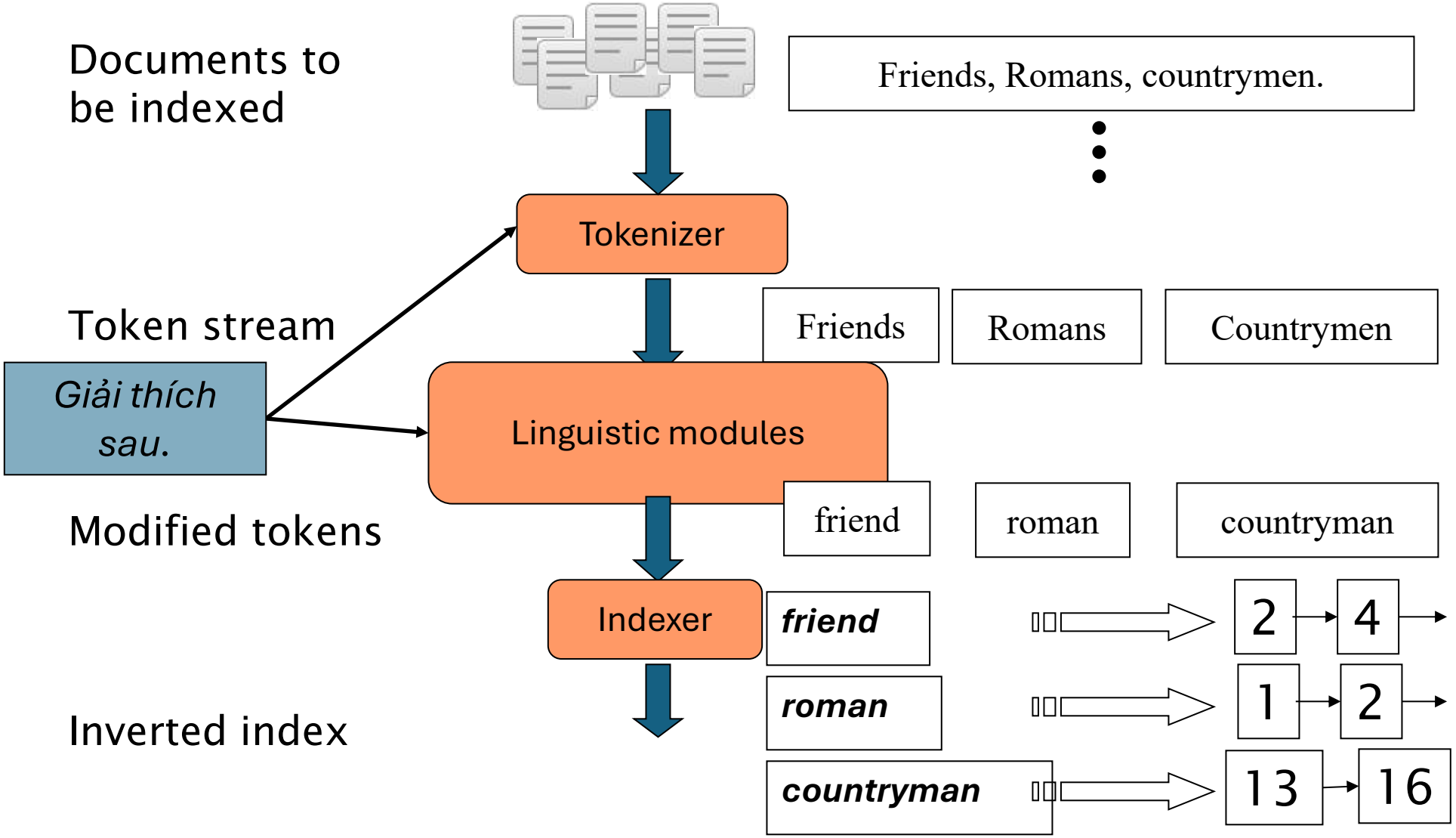
Chỉ mục ngược

- Cần các danh sách postings có kích thước thay đổi
  - Trên ổ đĩa, việc lưu các postings liên tục là phổ biến và hiệu quả nhất
  - Trong bộ nhớ, có thể sử dụng danh sách liên kết hoặc mảng có độ dài thay đổi được
    - Có sự đánh đổi giữa dung lượng lưu trữ và mức độ dễ dàng khi chèn dữ liệu



Sắp xếp theo **docID** (giải thích sau).

Xây dựng chỉ mục ngược



Các bước xử lý văn bản ban đầu

- Tokenization (Tách từ)
  - Cắt chuỗi ký tự thành các từ (token)
    - Xử lý các trường hợp như **“John’s”, a state-of-the-art solution**
- Normalization (Chuẩn hóa)
  - Đưa văn bản và từ trong truy vấn về cùng một dạng
    - Ví dụ: **U.S.A.** và **USA** được xem là giống nhau
- Stemming (Rút gọn từ)
  - Có thể muốn các dạng khác nhau của cùng một gốc từ được khớp
    - Ví dụ: **authorize, authorization**
- Stop words (Từ dừng)
  - Có thể loại bỏ các từ xuất hiện rất nhiều (hoặc không)
    - Ví dụ: **the, a, to, of**

Các bước lập chỉ mục - Chuỗi Token

- Tạo chuỗi các cặp (Token đã chỉnh sửa, ID tài liệu) từ văn bản gốc

Doc 1

I did enact Julius Caesar I was killed i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Các bước lập chỉ mục - Sắp xếp

- Sắp xếp danh sách theo **thuật ngữ** (alphabet) sau đó theo **docID**
- Đây là bước cốt lõi của việc lập chỉ mục

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

Các bước lập chỉ mục - Từ điển & Postings

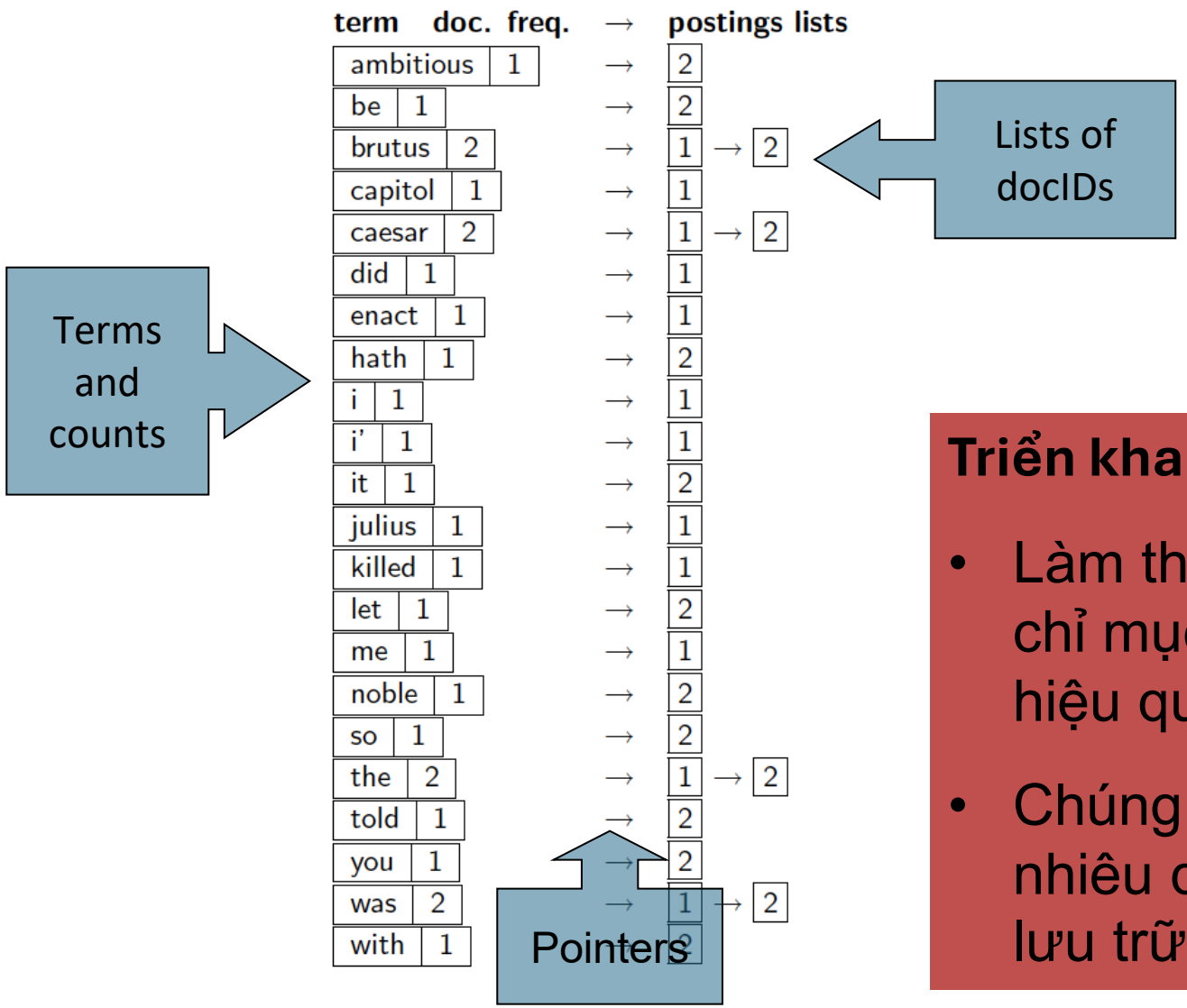
- Gộp các mục trùng lặp của cùng một thuật ngữ trong cùng một tài liệu
- Chỉ mục được tách thành từ điển và Postings
- Bổ sung thông tin tần suất của tài liệu

Tại sao lại là tần suất?
Sẽ thảo luận sau.

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
i	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

Lưu trữ và hiệu quả

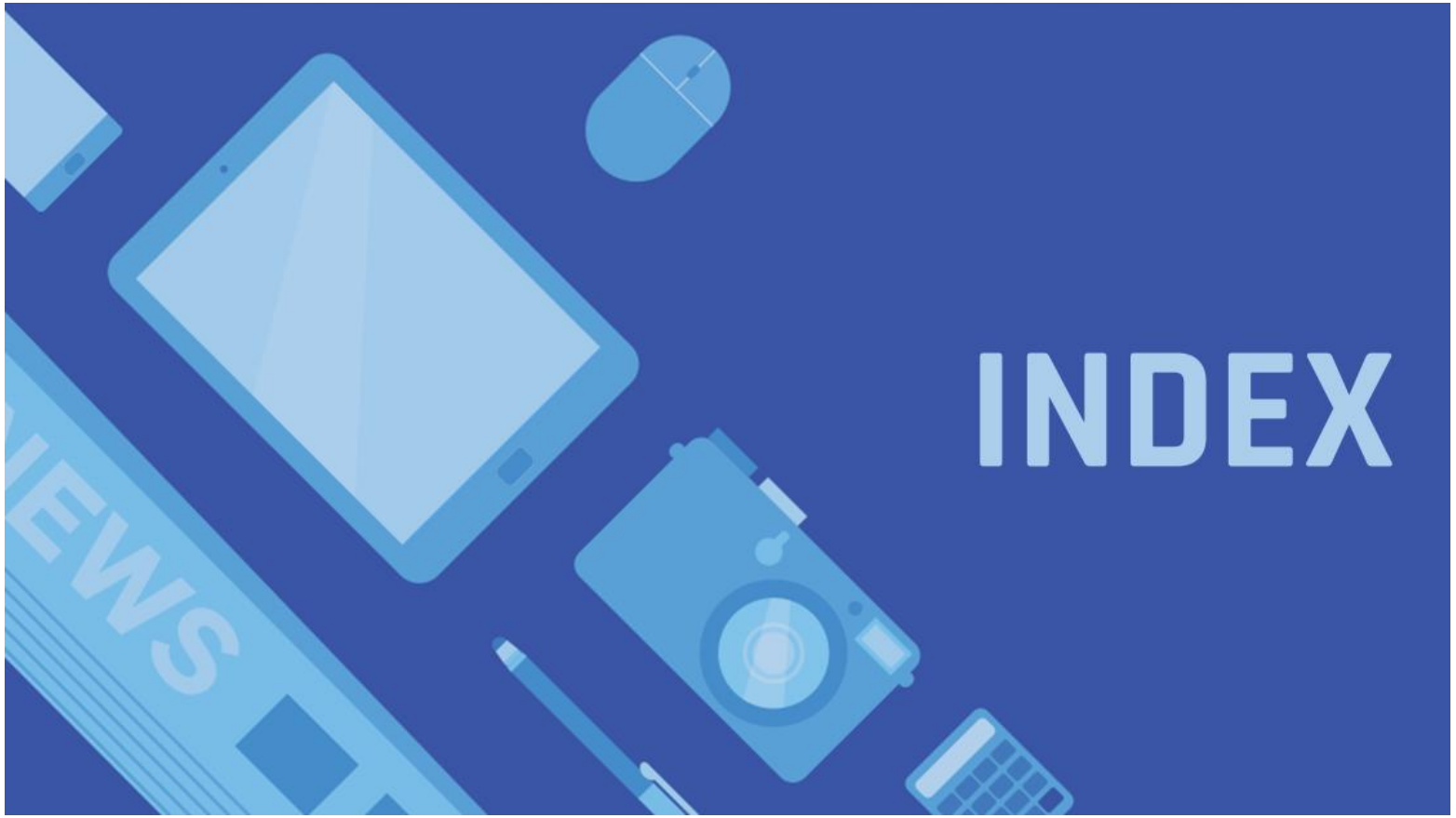


Triển khai hệ thống IR

- Làm thế nào để lập chỉ mục một cách hiệu quả?
- Chúng ta cần bao nhiêu dung lượng lưu trữ?

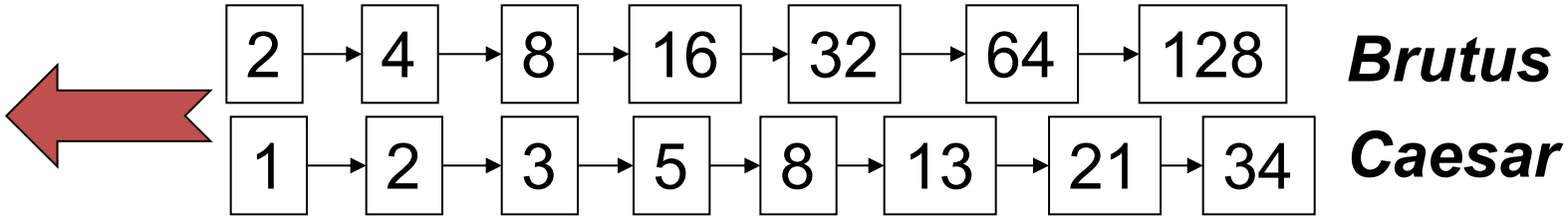
Chỉ mục đã xây dựng

- Làm thế nào xử lý một truy vấn?
  - Sau đó, những truy vấn nào có thể xử lý?



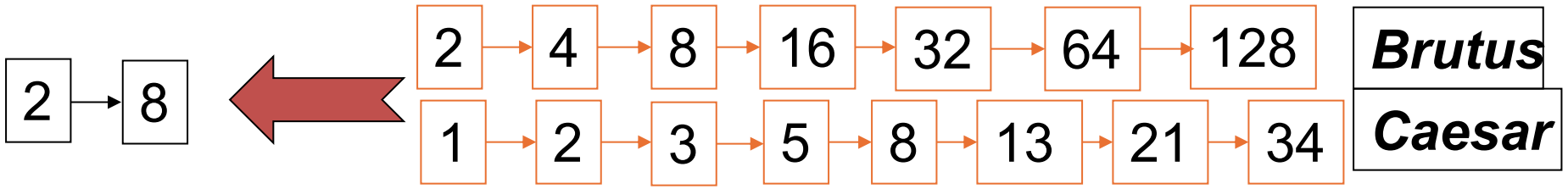
Xử lý truy vấn AND

- Ví dụ truy vấn: **Brutus** AND **Caesar**
  - Bước 1: Tìm **Brutus** trong từ điển, lấy danh sách postings
  - Bước 2: Tìm **Caesar**, lấy danh sách postings
  - Bước 3: Gộp hai postings bằng cách lấy giao các tập hợp tài liệu



Thuật toán gộp

- Duyệt qua hai postings đồng thời với thời gian tuyến tính O(x+y)
  - x, y là độ dài postings



- Chú ý: postings đã được sắp xếp theo docID

Mã giả

```
INTERSECT(p1, p2)
  answer ← ⟨ ⟩
  while p1 ≠ NIL and p2 ≠ NIL
  do if docID(p1) = docID(p2)
        then ADD(answer, docID(p1))
             p1 ← next(p1)
             p2 ← next(p2)
        else if docID(p1) < docID(p2)
                then p1 ← next(p1)
                else p2 ← next(p2)
  return answer
```

Mô hình truy vấn Boolean: Khớp chính xác

- Mô hình truy xuất Boolean cho phép đặt câu truy vấn dưới dạng một biểu thức Boolean
  - Truy vấn Boolean là các truy vấn sử dụng các toán tử AND, OR, NOT để kết hợp các từ khóa truy vấn
    - Mỗi tài liệu được xem như một tập hợp các từ
    - Tính chính xác: tài liệu thỏa mãn điều kiện hoặc không
  - Là một trong các mô hình đơn giản nhất để xây dựng một hệ thống IR
- Là công cụ truy xuất thương mại chủ yếu trong suốt 3 thập kỷ
- Nhiều hệ thống tìm kiếm đang dùng Boolean, ví dụ:
  - Email
  - Mục lục thư viện
  - macOS Spotlight

Ví dụ WestLaw

- http://www.westlaw.com/
- Dịch vụ tìm kiếm pháp lý thương mại lớn nhất (theo số người đăng ký trả phí) (bắt đầu năm 1975; bổ sung xếp hạng năm 1992; thêm tìm kiếm liên hợp mới năm 2010)
- Hàng chục terabyte dữ liệu; khoảng 700.000 người dùng
- Phần lớn người dùng vẫn sử dụng truy vấn Boolean
- Ví dụ truy vấn:
  - What is the statute of limitations in cases involving the federal tort claims act? (Thời hiệu khởi kiện trong các vụ việc liên quan đến Đạo luật khiếu nại bồi thường liên bang là gì?)
  - LIMIT! /3 STATUTE ACTION /S FEDERAL /2 TORT /3 CLAIM
    - /3 = trong phạm vi 3 từ, /S = trong cùng một câu

Ví dụ WestLaw

- Một ví dụ truy vấn khác:
  - Requirements for disabled people to be able to access a workplace (Các yêu cầu đối với người khuyết tật để có thể tiếp cận nơi làm việc)
  - disabl! /p access! /s work-site work-place (employment /3 place
- Lưu ý rằng DẤU CÁCH (SPACE) là phép tuyển (OR), không phải phép hội (AND)!
- Truy vấn dài, chính xác; toán tử khoảng cách; được phát triển dần theo từng bước; không giống tìm kiếm web
- Nhiều chuyên gia tìm kiếm vẫn ưa thích tìm kiếm Boolean
  - Biết chính xác mình đang nhận được kết quả gì
- Nhưng điều đó không có nghĩa là nó thực sự hoạt động tốt hơn…

Mô hình truy vấn Boolean: Gộp tổng quát hơn

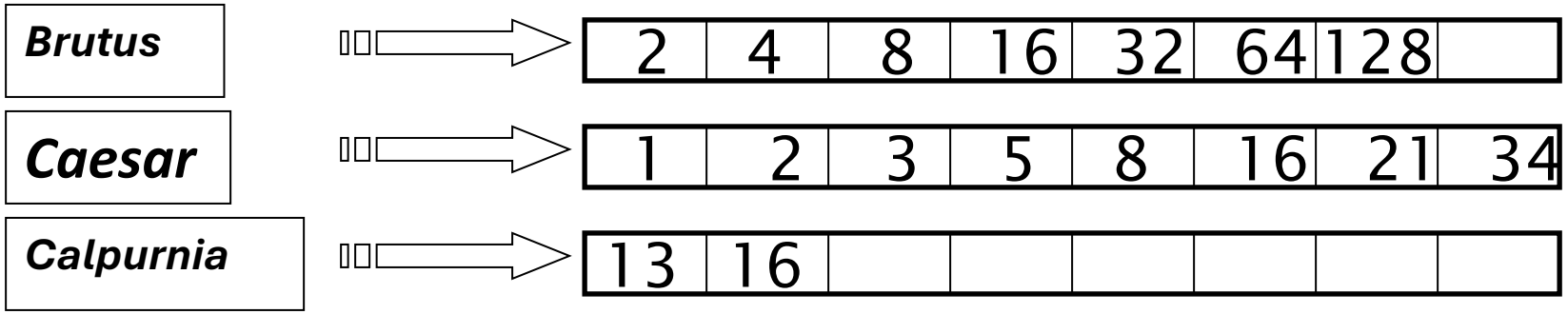
- Bài tập: Điều chỉnh thuật toán gộp cho các truy vấn:
  - **Brutus** AND NOT **Caesar**
  - **Brutus** OR NOT **Caesar**
- Chúng ta còn có thể thực hiện quá trình gộp với độ phức tạp thời gian O(x + y) không?
- Chúng ta có thể đạt được điều gì?

Mô hình truy vấn Boolean: Gộp tổng quát hơn

- Còn với một công thức Boolean bất kỳ thì sao?
  - (**Brutus** OR **Caesar**) AND NOT (**Antony** OR **Cleopatra**)
- Chúng ta có luôn có thể thực hiện merge trong thời gian "tuyến tính" không?
  - Tuyến tính theo cái gì?
- Chúng ta có thể làm tốt hơn không?

Tối ưu truy vấn

- Thứ tự tốt nhất để xử lý truy vấn là gì?
- Xét một truy vấn là phép AND của n thuật ngữ
- Với mỗi trong n thuật ngữ, lấy danh sách postings của nó, sau đó thực hiện AND các danh sách này với nhau



Query: ***Brutus*** *AND* ***Calpurnia*** *AND* ***Caesar***

Ví dụ tối ưu truy vấn

- Xử lý theo thứ tự tần suất tăng dần:
  - Bắt đầu với tập nhỏ nhất, sau đó tiếp tục thu hẹp dần

Đây là lý do giữ tần suất trong từ điển

Brutus	2	4	8	16	32	64	128	
Caesar	1	2	3	5	8	16	21	34
Calpurnia	13	16						

Execute the query as (***Calpurnia*** *AND* ***Brutus)*** *AND* ***Caesar***.

Bài tập

- Đề xuất một thứ tự xử lý truy vấn cho
  - (**tangerine** OR **trees**) AND (**marmalade** OR **skies**) AND (**kaleidoscope** OR **eyes**)
- Hai thuật ngữ nào nên được xử lý trước?

Term	Freq
eyes	213312
kaleidoscope	87009
marmalade	107913
skies	271658
tangerine	46653
trees	316812

Tối ưu hóa tổng quát hơn

- Ví dụ:
  - (**madding** OR **crowd**) AND (**ignoble** OR **strife**)
- Lấy tần suất tài liệu (doc. freq.) cho tất cả các thuật ngữ
- Ước lượng kích thước của mỗi phép OR bằng tổng tần suất tài liệu của các thuật ngữ trong đó (ước lượng)
- Xử lý theo thứ tự tăng dần của kích thước các phép OR

Tiêu chí	Tuần tự	Tối ưu
Thứ tự xử lý	Theo thứ tự viết	Theo kích thước ước lượng
Kích thước tập trung gian	Lớn	Nhỏ
Số phép so sánh	Nhiều	Ít
Bộ nhớ	Tốn	Tiết kiệm
Khả năng mở rộng	Kém	Tốt

Bài tập

- Hãy thử tính năng tìm kiếm tại

https://www.opensourceshakespeare.org/

- Ghi lại năm tính năng tìm kiếm mà bạn nghĩ hệ thống này có thể làm tốt hơn

Truy vấn cụm từ

- Trường hợp chúng ta muốn có khả năng trả lời các truy vấn như **“stanford university”** — được hiểu là một cụm từ liền nhau
- Vì vậy, câu **“I went to university at Stanford”** sẽ không được xem là một kết quả phù hợp
  - Khái niệm truy vấn cụm từ đã được chứng minh là dễ hiểu đối với người dùng; đây là một trong số ít các ý tưởng **“tìm kiếm nâng cao”** thực sự hoạt động hiệu quả
  - Trên thực tế, rất nhiều truy vấn là truy vấn cụm từ ngầm định
- Do đó, trong trường hợp này, việc chỉ lưu trữ các mục dạng:
  - **<term : docs>**

  không còn phù hợp

Cách tiếp cận ban đầu: Chỉ mục hai từ (Biword indexes)

- Lập chỉ mục cho mỗi cặp từ liên tiếp trong văn bản như một cụm từ
- Ví dụ, văn bản **"Friends, Romans, Countrymen"** sẽ tạo ra các biword sau:
  - **friends romans**
  - **romans countrymen**
- Mỗi biword này giờ đây được xem như một thuật ngữ trong từ điển
- Nhờ đó, việc xử lý truy vấn cụm từ gồm hai từ trở nên ngay lập tức và đơn giản

Truy vấn cụm từ dài hơn

- Các cụm từ dài hơn có thể được xử lý bằng cách chia nhỏ chúng ra
- Ví dụ, cụm từ **"stanford university palo alto"** có thể được tách thành **truy vấn Boolean trên các biword** như sau:
  - ***stanford university*** *AND* ***university palo*** *AND* ***palo alto***
- Tuy nhiên, nếu không có thông tin về tài liệu, chúng ta không thể xác minh rằng các tài liệu thỏa mãn truy vấn Boolean trên thực sự chứa cụm từ đầy đủ theo đúng thứ tự và liền kề

Có thể có dương tính giả

Biword mở rộng

- Phân tích văn bản được lập chỉ mục và thực hiện **gán nhãn từ loại** (*part-of-speech tagging – POST*)
- Chia các thuật ngữ vào các nhóm (ví dụ):
  - Danh từ (N)
  - Mạo từ / giới từ (X)
- Gọi bất kỳ chuỗi thuật ngữ nào có dạng **NX*N** là một **biword mở rộng** (trong đó X* nghĩa là có thể có **0 hoặc nhiều** từ loại X ở giữa)
  - Mỗi biword mở rộng như vậy sẽ được xem là một thuật ngữ trong từ điển
- Ví dụ: **catcher** *in the* **rye**

catcher	in	the	rye
N	**X**	**X**	**N**

- Xử lý truy vấn:
  - Phân tích truy vấn thành các **N** và **X**
  - Chia truy vấn thành các **biword mở rộng**
  - Tra cứu trong chỉ mục: **catcher rye**

Các vấn đề của chỉ mục hai từ

- Kết quả dương tính giả
- Chỉ mục phình to do từ điển trở nên lớn hơn
  - Không khả thi cho các cụm dài hơn hai từ, và ngay cả với hai từ thì kích thước cũng đã rất lớn
- Chỉ mục hai từ **không phải là giải pháp chuẩn** nhưng có thể được sử dụng như một phần của **chiến lược kết hợp**

Giải pháp 2: Chỉ mục vị trí

- Trong danh sách postings, lưu trữ vị trí xuất hiện của mỗi thuật ngữ trong tài liệu:

  <**term**, số lượng tài liệu chứa **term**;

  doc1: vị_trí1, vị_trí2, … ;

  doc2: vị_trí1, vị_trí2, … ;

  … >

Ví dụ chỉ mục vị trí

- Đối với truy vấn cụm từ, chúng ta sử dụng thuật toán trộn một cách đệ quy ở mức tài liệu
- Tuy nhiên, lúc này ta không còn chỉ xử lý phép so sánh bằng nhau nữa, mà phải xử lý mối quan hệ giữa các vị trí xuất hiện của các thuật ngữ (ví dụ: liền kề, đúng thứ tự).

<***be***: 993427;
1: 7, 18, 33, 72, 86, 231;
2: 3, 149;
4: 17, 191, 291, 430, 434;
5: 363, 367, …>

Tài liệu nào trong 1,2,4,5 có thể chứa cụm từ "***to be or not to be***"?

Xử lý một truy vấn cụm từ

- Trích xuất các mục trong **chỉ mục ngược** cho mỗi thuật ngữ khác nhau trong truy vấn: **to, be, or, not**
- Tiến hành trộn các danh sách **doc:position** của chúng để liệt kê tất cả các vị trí xuất hiện của cụm từ **"to be or not to be"**
  - **to:**
    - 2: 1,17,74,222,551;
    - 4: 8,16,190,429,433;
    - 7: 13,23,191; ...
  - **be:**
    - 1: 17,19;
    - 4: 17,191,291,430,434;
    - 5: 14,19,101; ...
- Phương pháp tổng quát tương tự cũng được áp dụng cho tìm kiếm theo độ gần (proximity searches)

Tìm kiếm theo độ gần

- LITMIT! /3 STATUTE /3 FEDERAL /2 TORT
  - Ở đây /k có nghĩa là “nằm trong phạm vi k từ”
- Rõ ràng, **chỉ mục vị trí** có thể dùng để xử lý các truy vấn kiểu này; còn **chỉ mục hai từ** thì không
- **Bài tập:** Hãy điều chỉnh thuật toán **trộn tuyến tính** của postings để xử lý **truy vấn theo độ gần**. Bạn có thể làm cho nó hoạt động với **mọi giá trị k** không?

Lời giải

```
PositionalIntersect(p1, p2, k)
answer ← ⟨ ⟩
while p1 ≠ NIL and p2 ≠ NIL
do if docID(p1) = docID(p2)
      then l ← ⟨ ⟩
           pp1 ← positions(p1)
           pp2 ← positions(p2)
           while pp1 ≠ NIL
           do while pp2 ≠ NIL
              do if |pos(pp1) − pos(pp2)| ≤ k
                    then Add(l, pos(pp2))
                    else if pos(pp2) > pos(pp1)
                            then break
                 pp2 ← next(pp2)
              while l ≠ ⟨ ⟩ and |l[0] − pos(pp1)| > k
              do Delete(l[0])
              for each ps ∈ l
              do Add(answer, ⟨docID(p1), pos(pp1), ps⟩)
              pp1 ← next(pp1)
           p1 ← next(p1)
           p2 ← next(p2)
      else if docID(p1) < docID(p2)
              then p1 ← next(p1)
              else p2 ← next(p2)
return answer
```

Kích thước chỉ mục vị trí

- Chỉ mục vị trí làm kích thước lưu trữ postings tăng lên đáng kể
  - Mặc dù các chỉ mục có thể được nén
- Nhưng chỉ mục vị trí hiện nay vẫn là giải pháp tiêu chuẩn được sử dụng, vì sức mạnh và tính hữu ích của truy vấn cụm từ và truy vấn độ gần … dù chúng được sử dụng một cách tường minh hay ngầm định trong hệ thống truy xuất xếp hạng

Kích thước chỉ mục vị trí

- Cần có một mục (entry) cho mỗi lần xuất hiện của thuật ngữ, chứ không chỉ một lần cho mỗi tài liệu
- Kích thước của chỉ mục phụ thuộc vào độ dài trung bình của tài liệu
  - Một trang web trung bình có dưới 1000 thuật ngữ
  - Các tài liệu của SEC, sách, thậm chí một số trường ca … có thể dễ dàng lên tới 100.000 thuật ngữ
- Xét một **thuật ngữ có tần suất xuất hiện 0,1%**

Document size	Postings	Positional postings
1000	1	1
100,000	1	100

Kinh nghiệm

- Chỉ mục vị trí có kích thước gấp 2–4 lần so với chỉ mục không có vị trí
- Kích thước của chỉ mục vị trí chiếm khoảng 35–50% dung lượng của văn bản gốc
  - Lưu ý: Tất cả các ước lượng trên chỉ đúng với các ngôn ngữ “giống tiếng Anh” (có khoảng trắng giữa từ, cấu trúc từ không quá phức tạp)

Các lược đồ kết hợp

- Hai cách tiếp cận này có thể được kết hợp với nhau một cách hiệu quả
  - Đối với một số cụm từ cụ thể (ví dụ: **"Michael Jackson", "Britney Spears"**), việc liên tục trộn các postings có vị trí là không hiệu quả
    - Điều này còn kém hiệu quả hơn nữa với những cụm như **"The Who"**
- Williams và cộng sự (2004) đã đánh giá một lược đồ lập chỉ mục hỗn hợp tinh vi hơn
  - Một hỗn hợp truy vấn web điển hình được thực thi chỉ mất 1/4 thời gian so với khi chỉ dùng chỉ mục vị trí
  - Đổi lại, nó cần thêm 26% dung lượng lưu trữ so với việc chỉ có chỉ mục vị trí

Bài tập

- Sau đây là một phần của chỉ mục vị trí với định dạng:
  - term: doc1: ⟨position1, position2, …⟩; doc2: ⟨position1, position2, …⟩; …

  angels: 2: ⟨36,174,252,651⟩; 4: ⟨12,22,102,432⟩; 7: ⟨17⟩;
  fools: 2: ⟨1,17,74,222⟩; 4: ⟨8,78,108,458⟩; 7: ⟨3,13,23,193⟩;
  fear: 2: ⟨87,704,722,901⟩; 4: ⟨13,43,113,433⟩; 7: ⟨18,328,528⟩;
  in: 2: ⟨3,37,76,444,851⟩; 4: ⟨10,20,110,470,500⟩; 7: ⟨5,15,25,195⟩;
  rush: 2: ⟨2,66,194,321,702⟩; 4: ⟨9,69,149,429,569⟩; 7: ⟨4,14,404⟩;
  to: 2: ⟨47,86,234,999⟩; 4: ⟨14,24,774,944⟩; 7: ⟨199,319,599,709⟩;
  tread: 2: ⟨57,94,333⟩; 4: ⟨15,35,155⟩; 7: ⟨20,320⟩;
  where: 2: ⟨67,124,393,1001⟩; 4: ⟨11,41,101,421,431⟩; 7: ⟨16,36,736⟩;

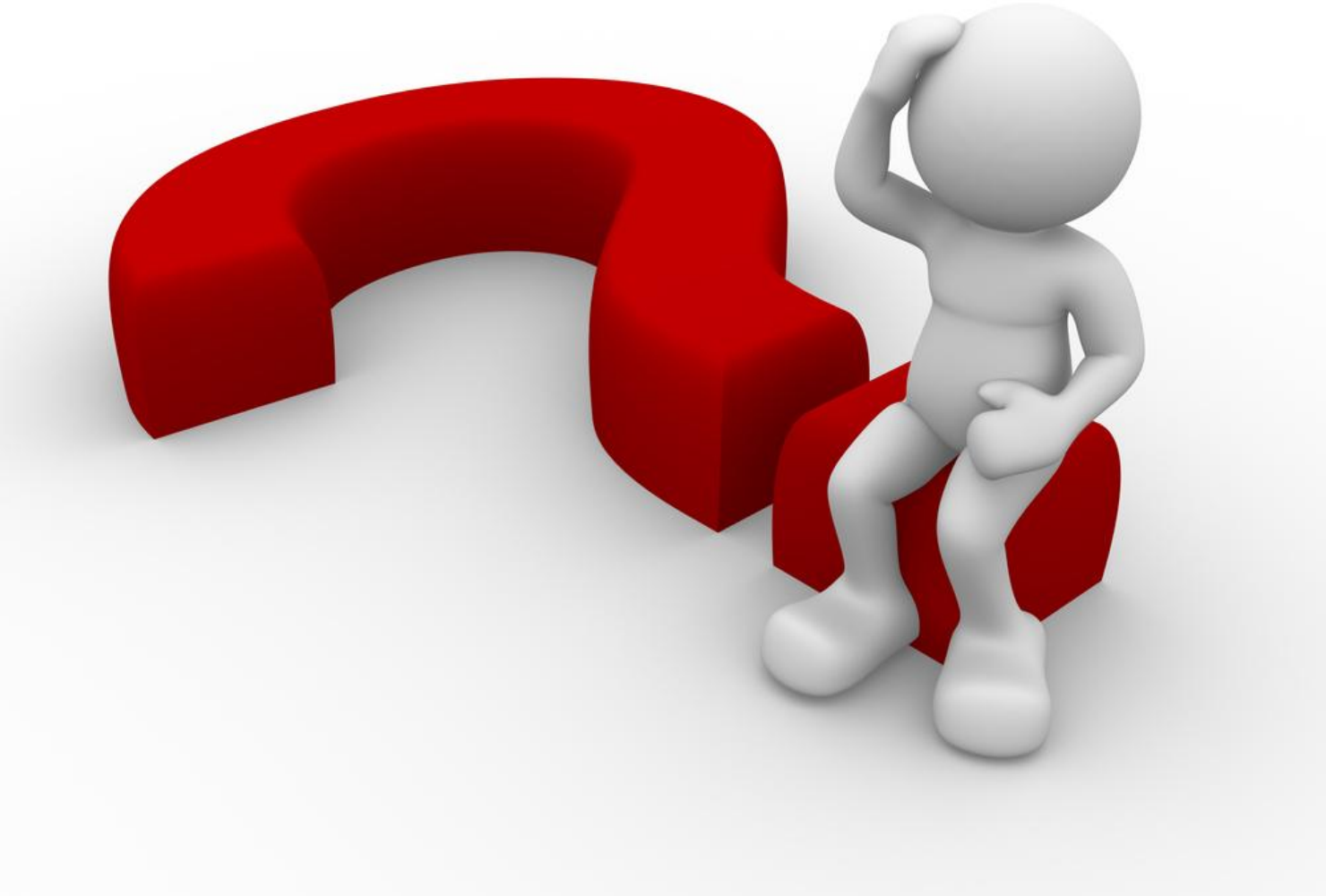
- Tài liệu nào (nếu có) khớp với các truy vấn sau, trong đó mỗi biểu thức trong dấu ngoặc kép là một truy vấn cụm từ?
  - **"fools rush in"**
  - **"fools rush in"** AND **"angels fear to tread"**

Bài tập

- Xét một phần của một chỉ mục vị trí, có định dạng:
  - word: document: ⟨position, position, …⟩; document: ⟨position, …⟩; …

  Gates: 1: ⟨3⟩; 2: ⟨6⟩; 3: ⟨2,17⟩; 4: ⟨1⟩;

  IBM: 4: ⟨3⟩; 7: ⟨14⟩;

  Microsoft: 1: ⟨1⟩; 2: ⟨1,21⟩; 3: ⟨3⟩; 5: ⟨16,22,51⟩;
- Toán tử **/k** trong truy vấn **word1 /k word2**, có nhiệm vụ tìm các lần xuất hiện của **word1** trong phạm vi **k** từ so với **word2** (ở cả hai phía), trong đó **k** là một số nguyên dương. Vì vậy, **k = 1** yêu cầu hai từ phải đứng liền kề nhau
  - Tìm tập các tài liệu thỏa mãn truy vấn **Gates /2 Microsoft**
  - Tìm các giá trị của **k** sao cho truy vấn **Gates /k Microsoft** trả về những tập tài liệu khác nhau

Hỏi đáp